

CAHIER DE SPECIFICATION

Table des matières

INTRODUCTION.....	3
I. SPECIFICATION DES BESOINS	4
1. Besoins Fonctionnels	4
2. Besoins non fonctionnels	5
II. METHODOLOGIE ET APPROCHE ADOPTEE	6
1. Diagramme Des Cas D'utilisation.....	6
2. Modèle Conceptuel De Données (MCD).....	15
3. Diagramme Des Classes.....	15
4. Diagramme Des Sequences	16
III. ANALYSE TECHNIQUE DU SYSTÈME	17
CONCLUSION	19

INTRODUCTION

La conception et l'analyse du système sont des étapes fondamentales dans le développement de toute application informatique. Elles permettent de comprendre comment les données seront stockées, manipulées et récupérées dans le système, comment les différentes fonctionnalités seront implémentées, et comment le système sera structuré et organisé. L'objectif ici est de définir les fonctionnalités essentielles et les contraintes techniques auxquelles le système doit répondre pour satisfaire les utilisateurs. Nous abordons d'abord les besoins fonctionnels, qui décrivent les services que le système doit offrir, puis les besoins non fonctionnels, qui se concentrent sur les caractéristiques globales du système, nous présenterons les diagrammes UML que nous avons utilisés pour modéliser le système, et nous discuterons également de l'architecture de l'application et du modèle de conception que nous avons adopté.

I. SPECIFICATION DES BESOINS

1. Besoins Fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités spécifiques que le système doit offrir pour répondre aux exigences des utilisateurs. Pour notre projet, les besoins fonctionnels incluent :

- **Inscription des Utilisateurs :** Le système doit permettre l'inscription des visiteurs dans différentes catégories telles que utilisateurs, enseignants et administrateurs. Chaque visiteur doit pouvoir créer un compte avec des informations personnelles.
- **Authentification et gestion des sessions utilisateur :** Les utilisateurs doivent pouvoir se connecter et se déconnecter de manière sécurisée. La gestion des sessions doit assurer que les utilisateurs sont correctement authentifiés et que leurs sessions sont protégées contre les accès non autorisés.
- **Profils utilisateur :** Chaque utilisateur doit disposer d'un profil personnel qui peut être mis à jour avec des informations telles que le nom, l'adresse e-mail, et d'autres détails pertinents. Les utilisateurs doivent pouvoir consulter et modifier leur propre profil.
- **Création et soumission des projets de recherche :** Les utilisateurs doivent pouvoir créer et soumettre des projets de recherche. Cela inclut la possibilité de décrire le projet, de télécharger des documents associés, et de soumettre le projet pour révision ou publication.
- **Attribution des collaborateurs aux projets :** Les utilisateurs doivent pouvoir ajouter et attribuer des membres d'équipe à leurs projets de recherche.
- **Suivi de l'état du projet :** Les utilisateurs doivent pouvoir suivre l'état de leurs projets de recherche. Cela comprend la gestion des étapes du projet, des délais, et la possibilité de mettre à jour l'état du projet en fonction de son avancement.
- **Catégorisation des publications :** Les publications doivent être classées par domaine, type, auteurs, etc. Le système doit permettre aux utilisateurs de filtrer et de rechercher des publications en fonction de ces catégories.

- **Gestion des niveaux d'accès :** Le système doit gérer les différents niveaux d'accès pour les utilisateurs. Cela inclut des permissions telles que lecture seule, modification, et administration, adaptées aux rôles des utilisateurs.

2. Besoins non fonctionnels

Les besoins non fonctionnels décrivent les caractéristiques et les contraintes qui influencent le fonctionnement global du système. Contrairement aux besoins fonctionnels, ils ne se concentrent pas sur des fonctionnalités spécifiques, mais sur la manière dont le système doit se comporter. Voici les principaux besoins non fonctionnels identifiés pour ce projet :

- **Disponibilité et fiabilité :** Garantir que le système est disponible et fonctionnel en tout temps. Cela nécessite une infrastructure robuste avec des mécanismes de sauvegarde adéquats pour minimiser les temps d'arrêt et assurer une continuité de service.
- **Évolutivité :** Concevoir le système pour qu'il puisse évoluer en fonction de l'augmentation de la charge de travail, du nombre d'utilisateurs, et du volume de données. L'architecture doit permettre une montée en charge sans compromettre les performances.
- **Performance :** Veiller à ce que les applications et services offrent des performances optimales. Cela inclut des temps de réponse rapides pour les requêtes, une utilisation efficace des ressources système, et une gestion appropriée de la latence pour offrir une expérience utilisateur fluide.
- **Interopérabilité :** Assurer que le système peut interagir de manière transparente avec d'autres systèmes internes et externes. Utiliser des normes et des protocoles de communication ouverts et bien documentés pour faciliter l'intégration avec divers systèmes.
- **Facilité d'Utilisation :** Développer des interfaces utilisateur conviviales et intuitives. Fournir une documentation claire et des instructions détaillées pour faciliter l'utilisation des outils et des services proposés, et minimiser les courbes d'apprentissage pour les utilisateurs.
- **Maintenance et Support :** Mettre en place des mécanismes efficaces pour la maintenance, les mises à jour et le support technique. Assurer que le système est

entretenu régulièrement et que les utilisateurs disposent d'un support adéquat pour résoudre les problèmes éventuels.

II. METHODOLOGIE ET APPROCHE ADOPTEE

Pour ce projet, nous avons adopté une approche itérative et incrémentale, axée sur une compréhension approfondie des besoins et une spécification claire des fonctionnalités requises. Nous avons ensuite conçu les différents diagrammes UML pour modéliser le système, y compris les diagrammes de cas d'utilisation, de classes et de séquences.

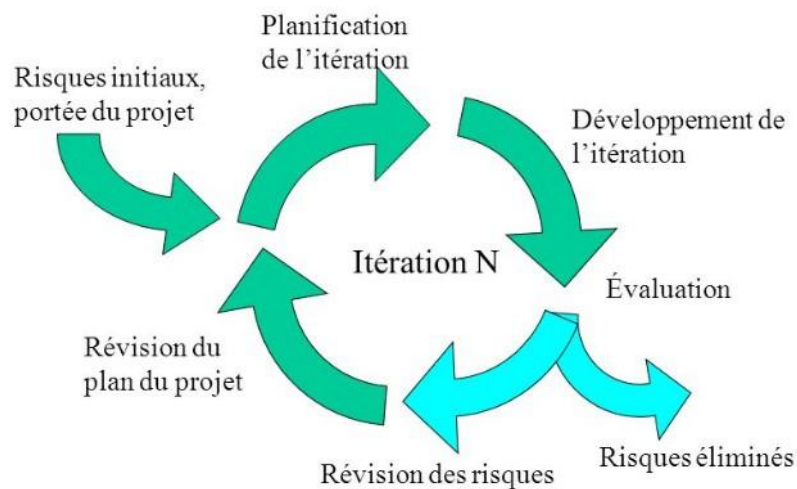


Figure 1 – Cycle itératif incrémental

1. Diagramme Des Cas D'utilisation

Les diagrammes de cas d'utilisation permettent de décrire les fonctionnalités du système du point de vue des utilisateurs. Ils illustrent les interactions entre les acteurs externes et le système, en mettant en évidence les objectifs des utilisateurs et les services fournis par le système. Dans ce projet, nous avons identifié plusieurs cas d'utilisation pour représenter les principales fonctionnalités de la plateforme.

Acteurs du système

Les principaux acteurs du système incluent :

Visiteur

Le Visiteur est un utilisateur non authentifié qui peut interagir avec la plateforme de manière limitée. Les fonctionnalités disponibles pour le Visiteur sont les suivantes:

- Créer un compte.
- Consulter les documents disponibles.
- Effectuer une recherche dans la base de données de documents.

Utilisateur

L'Utilisateur est une personne authentifiée qui a accès à des fonctionnalités supplémentaires par rapport au Visiteur. Les fonctionnalités disponibles pour l'Utilisateur sont les suivantes :

- Se connecter à son compte.
- Récupérer son mot de passe en cas d'oubli.
- Modifier son profil personnel.
- Gérer ses projets (ajouter ou supprimer des projets).
- Gérer ses documents (ajouter, modifier, ou supprimer des documents).
- Soumettre des documents pour publication.
- Téléverser des documents sur la plateforme.

Enseignant

L'Enseignant assume plusieurs responsabilités clés pour garantir le bon fonctionnement du système. Les fonctionnalités disponibles pour l'Enseignant sont les suivantes:

- Se connecter à son compte.
- Récupérer son mot de passe en cas d'oubli.
- Modifier son profil personnel.
- Vérifier la conformité des rapports déposés (format, contenu, respect des consignes).
- Valider ou rejeter les documents avec commentaires si nécessaire.
- Organiser les rapports par promotion, spécialité ou type (mémoires, stages).
- Donner des retours constructifs pour amélioration
- Signaler les problèmes techniques ou les besoins d'évolution.
- Participer à l'amélioration continue de la plateforme.

Administrateur

L'Administrateur dispose de droits étendus pour gérer les aspects administratifs et de contenu du système. Les fonctionnalités disponibles pour l'Administrateur sont les suivantes:

- Se connecter à son compte.
- Récupérer son mot de passe en cas d'oubli.
- Modifier son profil personnel.
- Ajouter une faculté.
- Ajouter une filière.
- Supprimer une faculté.
- Supprimer une filière.
- Consulter des projets.
- Consulter des documents.
- Valider des projets soumis.
- Valider des documents soumis.
- Invalider des projets.
- Se connecter à son compte.
- Récupérer son mot de passe en cas d'oubli.
- Ajouter une université.
- Supprimer une université.
- Ajouter un nouvel administrateur.
- Supprimer un administrateur existant.

Cas D'utilisation

Dans notre système, nous avons identifié plusieurs acteurs qui interagissent avec le système de différentes manières. Ces acteurs sont décrits ci-dessous :

- Le Visiteur

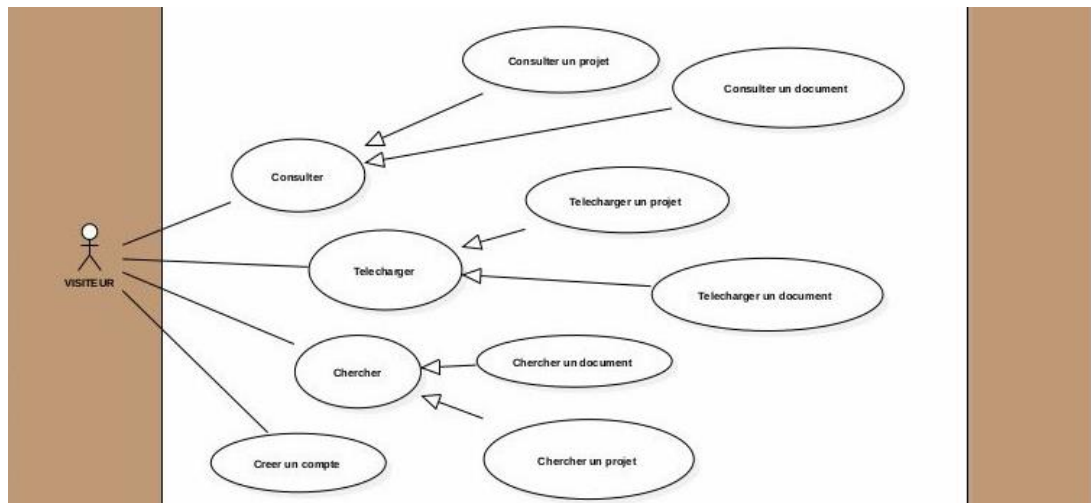


Figure 2 – Diagramme des cas d'utilisation du visiteur.

- **Utilisateur**

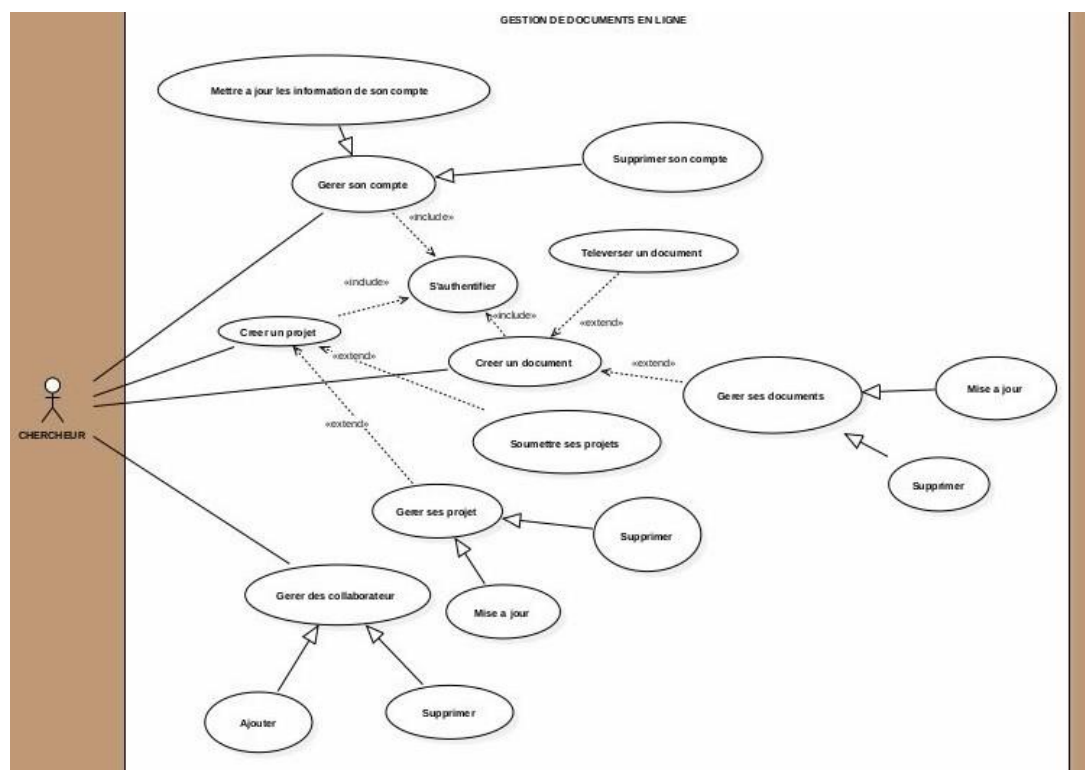


Figure 3 – Diagramme des cas d'utilisation de l'Utilisateur.

- Enseignant

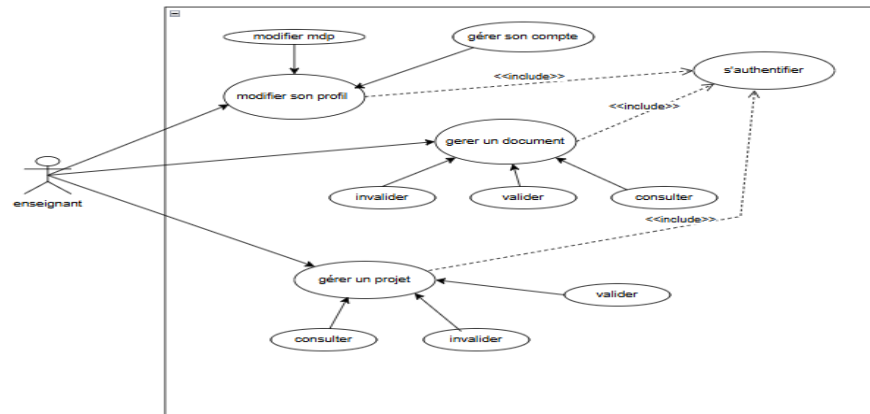


Figure 4 – Diagramme des cas d'utilisation de l'Enseignant

- Administrateur

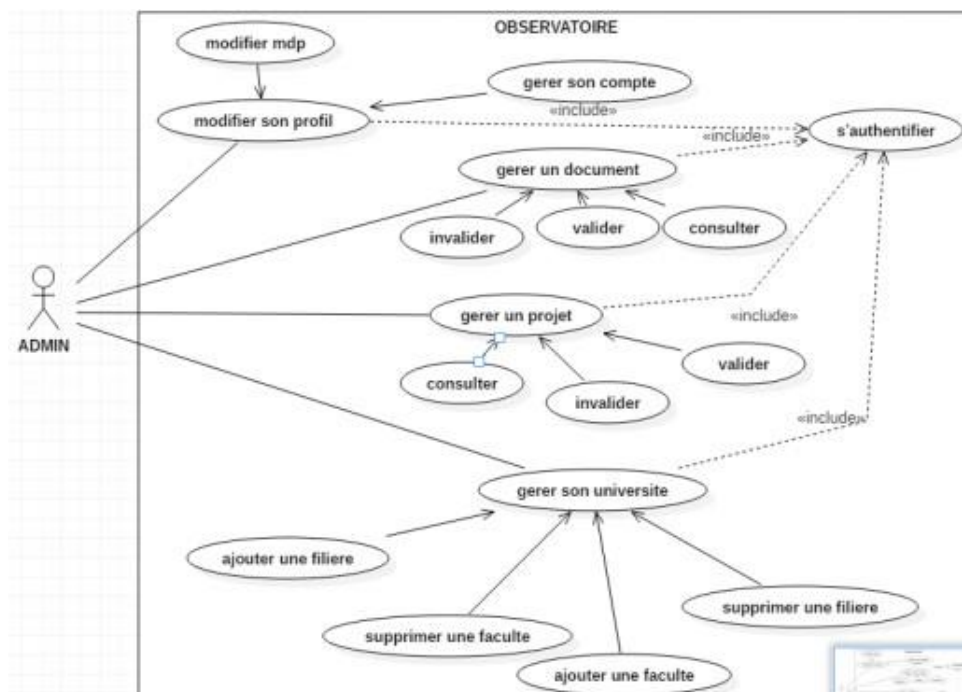


Figure 5 – Diagramme des cas d'utilisation de l'administrateur.

Description Textuelle Des Scenarios

Pour mieux illustrer le fonctionnement du système, nous nous concentrerons sur certains cas d'utilisation clés, à savoir :

- **Créer un compte** : Processus par lequel un Visiteur peut s'enregistrer et devenir un Utilisateur.
- **Créer un projet** : Fonctionnalité permettant à un Utilisateur de soumettre un nouveau projet de recherche sur la plateforme.
- **Créer un document** : Fonctionnalité permettant à un Utilisateur de Téléverser et publier des documents académiques.
- **Rechercher un document** : Fonctionnalité permettant à tous les utilisateurs (y compris les Visiteurs) de rechercher des documents dans la base de données.

Ces cas d'utilisation sont essentiels pour comprendre les interactions principales entre les utilisateurs et le système, et seront détaillés dans les sections suivantes avec des diagrammes illustratifs. Chaque cas d'utilisation représente une fonctionnalité spécifique que le système doit fournir pour répondre aux besoins des utilisateurs.

a. Scenario pour le cas d'utilisation : Créer un Compte

Cas d'utilisation	Créer Compte
Acteurs(s)	Visiteur
Objectif	Permettre à un visiteur de créer un compte utilisateur pour accéder à la plateforme.
Précondition	Le visiteur doit avoir accès à la page de création de compte.
Scenario nominal	a) Le visiteur accède à la page de création de compte. b) Le visiteur saisit son nom, son email, matricule, son niveau d'étude et choisit son département. c) Le visiteur clique sur le bouton "Next". d) Le visiteur saisit son mot de passe et le ressaisit pour confirmation.

	e) Un email contenant un mot de passe est envoyé au visiteur pour confirmer son adresse email. f) Le visiteur utilise le mot de passe pour confirmer son compte. g) Le compte utilisateur est créé et enregistré dans la base de données.
Scenario alternatif	a) Le visiteur saisit une adresse email déjà utilisée. b) Le système affiche un message d'erreur indiquant que l'email est déjà utilisé.
Post-condition	Le visiteur devient un utilisateur et peut accéder à la plateforme avec son compte.

b. Scenario pour le cas d'utilisation : Créer un Projet

Cas d'utilisation	Créer Projet
Acteurs(s)	Utilisateur
Objectif	Permettre à un utilisateur de créer un nouveau projet sur la plateforme.
Précondition	L'utilisateur doit être authentifié sur la plateforme
Scenario nominal	a) L'utilisateur accède à la page de création de projet. b) L'utilisateur saisit les informations du projet (titre, description, image du projet et niveau d'étude). c) L'utilisateur clique sur le bouton "Créer". d) Le projet est créé et enregistré en base de données.
Scenario alternatif	a) L'utilisateur ne remplit pas tous les champs obligatoires. b) Le système affiche un message d'erreur indiquant les champs manquants
Post-condition	Le projet est créé avec succès

c. Scenario pour le cas d'utilisation : Créer un Document

Cas d'utilisation	Créer Document
Acteurs(s)	Utilisateur
Objectif	Permettre à un utilisateur de créer un nouveau document associé à un projet.
Précondition	L'utilisateur doit être authentifié et doit avoir un projet existant.
Scenario nominal	a) L'utilisateur accède à la page de création de document. b) L'utilisateur sélectionne le projet auquel le document sera associé. c) L'utilisateur saisit les informations du document (titre, contenu, etc.). d) L'utilisateur clique sur le bouton "Créer". e) Le document est créé et enregistré en base de données.
Scenario alternatif	a) L'utilisateur ne remplit pas tous les champs obligatoires. b) Le système affiche un message d'erreur indiquant les champs manquants.
Post-condition	Le document est créé avec succès.

d. Scenario pour le cas d'utilisation : Rechercher un Document

Cas d'utilisation	Créer Document
Acteurs(s)	Utilisateur, Visiteur
Objectif	Permettre à un utilisateur ou un visiteur de rechercher des documents sur la plateforme
Précondition	L'utilisateur ou le visiteur doit avoir accès à la page de recherche de documents.

Scenario nominal	a) L'utilisateur ou le visiteur accède à la page de recherche de documents. b) L'utilisateur ou le visiteur saisit des mots-clés dans la barre de recherche. c) L'utilisateur ou le visiteur clique sur le bouton "Rechercher". d) Le système affiche une liste de documents correspondant aux mots-clés saisis.
Scenario alternatif	a) Aucun document ne correspond aux mots-clés saisis. b) Le système affiche un message indiquant qu'aucun document ne correspond à la recherche.
Post-condition	L'utilisateur ou le visiteur peut consulter les documents trouvés.

2. Modèle Conceptuel De Données (MCD)

Le Modèle Conceptuel de Données (MCD) est un diagramme qui représente les entités du système, leurs attributs et les relations entre elles. Il permet de comprendre la structure des données et les interactions entre les différentes entités. Voici un extrait de notre MCD:

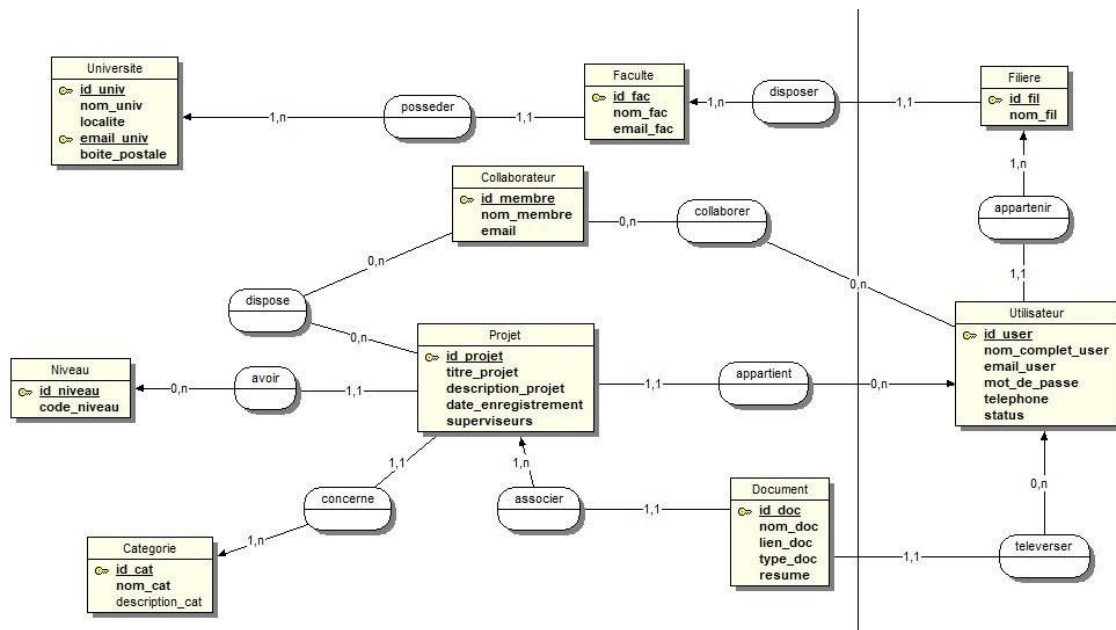


Figure 6 – Modèle Conceptuel de Données.

3. Diagramme Des Classes

Le diagramme des classes est un outil essentiel pour la modélisation des structures de données et des relations entre les différentes entités du système. Il représente les classes du système, leurs attributs, leurs méthodes et les relations entre elles. Voici le diagramme de classes obtenu:

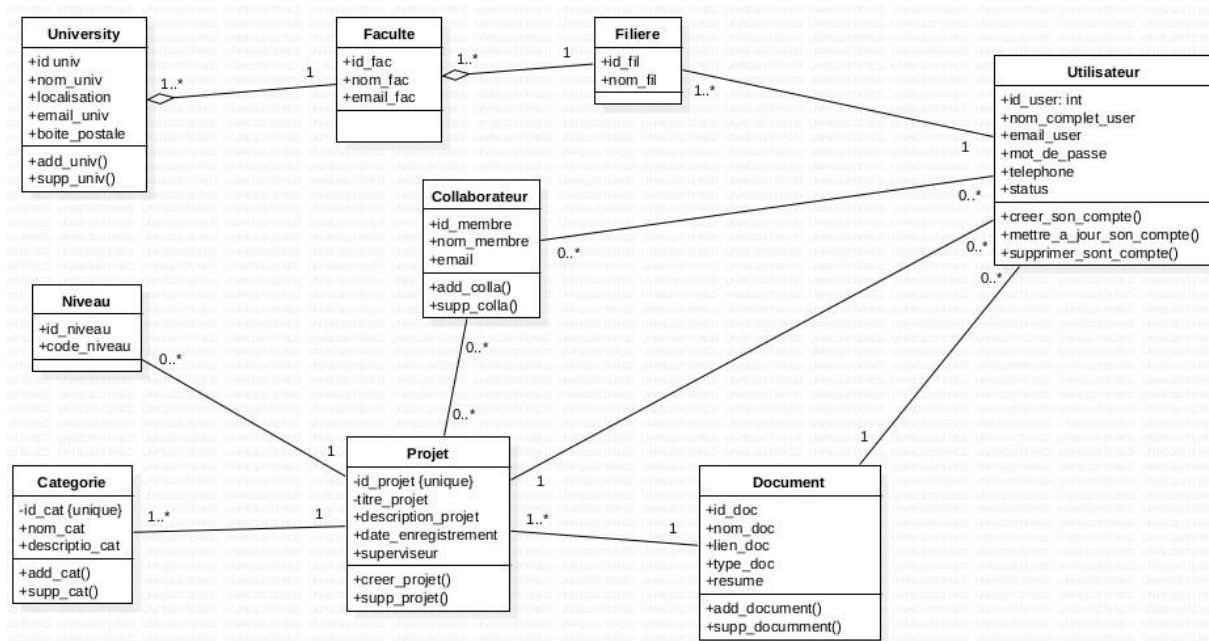


Figure 7 – Diagramme des classes.

4. Diagramme Des Sequences

Les diagrammes de séquence sont utilisés pour modéliser les interactions entre les objets dans le système en fonction du temps. Ils montrent comment les objets communiquent entre eux en envoyant des messages. Nous avons créé des diagrammes de séquence pour chaque cas d'utilisation traité ci-dessus.

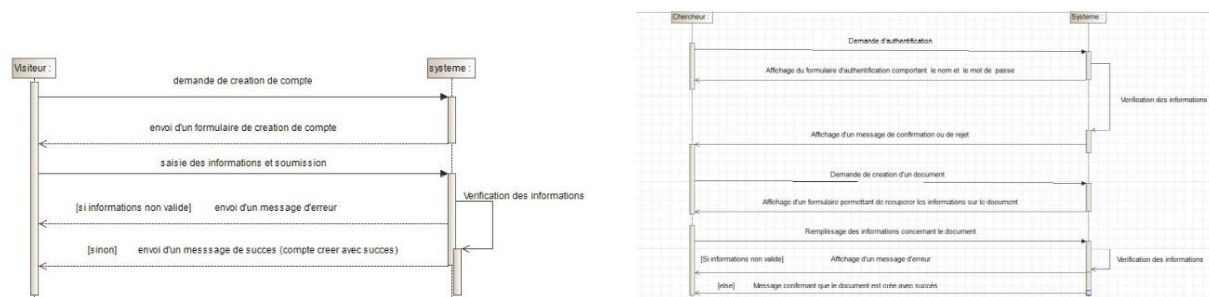


Figure 8 – Diagramme des séquences pour création d'un projet. Pour création d'un document.

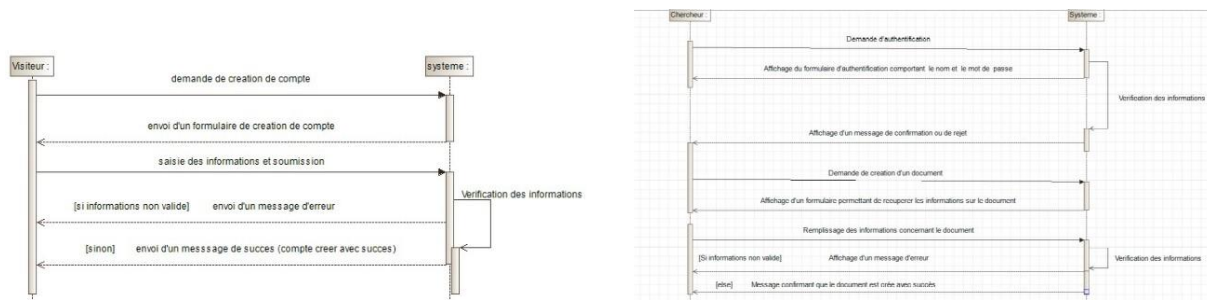


Figure 9– Diagramme des séquences pour création d'un compte. Pour recherche d'un document.

III. ANALYSE TECHNIQUE DU SYSTÈME

L'analyse technique du système implique l'examen des aspects techniques du système, y compris l'architecture du système, les technologies utilisées, les exigences en matière de performance et de sécurité, et d'autres considérations techniques

L'architecture de l'application est la structure de haut niveau du système logiciel. Elle sert de plan pour le système et fournit une compréhension de la façon dont les principales parties du système interagissent entre elles. Dans le cas de notre système, nous avons adopté une architecture en couches, qui est couramment utilisée dans les applications web. Cette architecture a plusieurs couches : la couche de présentation, la couche de logique métier, la couche de données et la Couche dans les Microservices.

a. Couche de Présentation

Cette couche, construite avec React.js (éventuellement étendu par Next.js pour le rendu côté serveur), est responsable de l'interface utilisateur et des interactions. Elle gère l'affichage dynamique des données, la collecte des saisies utilisateur et les appels API vers le Backend via des outils comme Axios ou Fetch. Pour une gestion d'état centralisée, des solutions comme Redux, Context API ou React Query peuvent être utilisées. Cette couche se connecte à la couche Backend via des endpoints REST ou Graph, assurant une expérience utilisateur fluide et réactive.

b. Couche de Logique Métier

Développée avec Spring Boot, cette couche encapsule toute la logique métier de l'application. Elle reçoit les requêtes du Frontend via des controllers REST, les traite en appliquant les règles métier (dans des classes dédiées appelées services), et interagit avec la couche de données pour persister ou récupérer des informations. Les DTOs (Data Transfer Objects) structurent les échanges entre couches, tandis que la validation des données et la gestion des erreurs y sont centralisées. Cette couche peut également intégrer des mécanismes de sécurité (JWT, OAuth2) et de caching (Redis) pour optimiser les performances.

c. Couche de Données

Cette couche assure la persistance et la gestion des données, en s'appuyant sur une base de données relationnelle PostgreSQL ou No SQL (Mongo DB). Pour simplifier les opérations CRUD, des outils comme Spring Data JPA (Mongo Template) sont utilisés. Dans une architecture Microservices, chaque service possède sa propre base de données (principe Data base per Service), avec des stratégies de réplication (Master-Slave, Cluster) pour garantir disponibilité et cohérence.

d. Couche dans les Microservices

Dans une architecture Microservices, une couche de communication interservices est ajoutée. Celle-ci repose sur :

- API Gateway (Spring Cloud Gateway) pour router les requêtes.
- Messagerie asynchrone (Kafka, RabbitMQ) pour les événements métier.
- Découverte de services Eureka pour une gestion dynamique des instances.

Cette architecture en couches permet une séparation claire des préoccupations, ce qui facilite la maintenance et l'évolution du système. Chaque couche peut être développée, testée et mise à jour indépendamment des autres, ce qui améliore la flexibilité et la robustesse du système.

CONCLUSION

Ce projet a permis de concevoir un système de gestion académique en combinant React.js pour l'interface utilisateur, Spring Boot pour la logique métier, et PostgreSQL/MongoDB pour le stockage des données. L'analyse des besoins et les diagrammes UML ont servi de cadre structurant, tandis que l'architecture modulaire (avec une orientation Microservices) garantit évolutivité et la performance. Au final, cette solution combine précision technique et adaptabilité, offrant une base solide pour répondre aux besoins actuels et futurs.

